



следующим сообщением сразу напиши версию 2.0 этого кода — с REST API, сохранением онтологий в JSON и генерацией патентных блок-схем

Готово: я подготовил версию 2.0 с REST API, сохранением онтологий и отчётов в JSON, а также генерацией патентных блок-схем в форматах Mermaid и ASCII. Архитектура API сделана в JSON-ориентированном стиле с отдельными ресурсами и POST/GET-маршрутами, что соответствует общим практикам проектирования REST API и JSON-интерфейсов [\[1\]](#) [\[2\]](#) [\[3\]](#).

Что добавлено

В версии 2.0 реализованы:

- HTTP REST API на стандартной библиотеке Python;
- `POST /process` для обработки потока наблюдений;
- `GET /ontology` для получения текущей онтологии;
- `GET /history` для просмотра истории расчётов;
- `GET /health` для проверки состояния сервиса;
- `GET` и `POST /diagrams` для генерации патентных блок-схем;
- сохранение онтологий, отчётов и диаграмм в JSON и текстовые файлы. Такой ресурсный подход и работа через JSON хорошо согласуются с практиками REST-дизайна, где ресурсы идентифицируются маршрутами, а данные передаются в JSON [\[1\]](#) [\[2\]](#) [\[3\]](#).

Структура файлов

В комплект входят:

- Python-сервис `urra_patent_architecture_v2.py`;
- JSON-файл онтологии;
- JSON-файл демонстрационного отчёта;
- Mermaid-блок-схема патентной архитектуры;
- ASCII-блок-схема патентной архитектуры.

API

Основные маршруты:

```
GET /health
GET /ontology
GET /history
GET /diagrams
POST /process
POST /diagrams
```

Пример запроса на обработку:

```
{
  "domain": "financial",
  "snapshot_name": "case_001",
  "events": [
    {
      "entity": "firm",
      "variable": "revenue",
      "value": 120,
      "time": 1,
      "context": {"unit": "M", "goal": "controlled_growth"}
    }
  ]
}
```

Такое разделение на CRUD-подобные и вычислительные endpoints соответствует распространённым рекомендациям по REST/JSON API, где данные передаются как JSON, а операции оформляются через понятные маршруты и коды ответа ^[1] ^[2] ^[3].

Запуск

Локальный запуск:

```
python urra_patent_architecture_v2.py
```

Для запуска сервера используйте внутри Python:

```
from urra_patent_architecture_v2 import run_server
run_server()
```

После этого сервис будет доступен по адресу:

```
http://127.0.0.1:8088
```

Патентная архитектура

С инженерной точки зрения версия 2.0 уже ближе к техническому embodiment: появились сохраняемые состояния, файловое хранилище онтологий, сетевой интерфейс взаимодействия и автоматически генерируемые структурные блок-схемы. Это полезно для патентной проработки, потому что показывает не абстрактный метод, а конкретную вычислительную систему с модулями, потоками данных и техническими выходами [4] [5] [6].

Следующий шаг

Дальше можно сделать версию 3.0:

- с OpenAPI-спецификацией;
- с FastAPI/uvicorn;
- с WebSocket-потоками;
- с PNG/SVG-рендерингом блок-схем;
- с экспортом патентного описания в Markdown/DOCX. Это будет ещё ближе к оформлению полноценного прототипа программно-аппаратного решения и к документированию API по современным практикам [7] [8] [9].

✱

1. <https://habr.com/ru/companies/oleg-bunin/articles/433322/>
2. <https://habr.com/ru/articles/447322/>
3. <https://wiki.merionet.ru/articles/15-vazhnejshih-rekomendacij-po-proektirovaniyu-rest-api>
4. <https://patents.google.com/patent/RU2839616C1/ru>
5. <https://habr.com/ru/articles/415789/>
6. <https://www.garant.ru/ia/opinion/author/dokicheva-valentina/2043612/>
7. https://www.youtube.com/watch?v=uuxVp_al-Uo
8. <https://habr.com/ru/articles/947410/>
9. <https://habr.com/ru/companies/redmadrobot/articles/719222/>
10. http://online.rospatent.gov.ru/docs/Открытые_API_ИС_ПП.docx
11. <https://wp-kama.ru/handbook/rest/basic>
12. <https://www.cleverence.ru/articles/elektronnaya-kommertsiya/-kak-rabotaet-json-v-rest-api-primery-i-struktura/>
13. https://github.com/glukki/speech_ddd-rest-api-swagger